E1CSA313 DESIGN PROJECT-1 · ALLIANCE UNIVERSITY

Chain of Record

Blockchain-Based Verifiable Academic Credential System
Project file guide: every folder, every file and how they work together

Team	Manu M P, Tarun Kumar S, Anilkumar U K, Pruthviraj D N
Mentor	Dr. Saravanan P
Website	https://blockchain-ecru-three.vercel.app
Smart contract	Ethereum Sepolia, 0xA34BE0F943de87b2414da6Ea44A49c2bb8d5B37a
Prepared	2 November 2026, for the final review

Contents

1. What the project does
2. System architecture
3. Folder map
4. Workflow A: loading the dataset onto the chain
5. Workflow B: how one block is created (consensus)
6. Workflow C: issuing and revoking a certificate
7. Workflow D: a student downloads the marks card
8. Workflow E: verifying a certificate photo
9. Workflow F: tamper detection and repair
10. Workflow G: public anchor on Ethereum Sepolia
11. Workflow H: sign-in and accounts
12. Files: the blockchain (`backend/blockchain`)
13. Files: the API server (`backend/server`)
14. Files: database migrations
15. Files: scripts (commands)
16. Files: tests and results
17. Files: smart contract, data and settings
18. Files: the website (`src`, `public`, `api`)
19. Root files
20. API endpoints
21. Database tables
22. Commands you can run
23. Measured results
24. Known limits
25. Generated folders and the archive

1. What the project does

Input: a photo of a certificate (marks card). **Output: VALID**, **INVALID** or **REVOKED**.

Universities issue certificates. Each certificate's fingerprint (SHA-256 of its data) is recorded on a blockchain that the team wrote from scratch. An employer photographs the certificate; the website reads it, finds the certificate on the chain, checks that every printed detail matches the university's record and that the record's hash matches the chain, and shows the verdict.

106 blocks on the chain	**1,707** signed transactions	**1,604** certificates recorded (86 revoked)
3 validator nodes, majority of 2 must agree	**666 / 666** labelled verification requests correct	**90 / 90** forged test photos rejected

Technology used

Part	Technology
Website	React 18, Vite, Tailwind CSS, hosted on Vercel
Photo reading	Tesseract.js (OCR) and qr-scanner, both running inside the browser
API server	Node.js, Express, JWT logins, scrypt password hashing, helmet, rate limits
Blockchain	Own implementation: SHA-256, Merkle trees, ECDSA secp256k1 signatures, proof of work, proof of authority with 3 validators
Database	PostgreSQL on Supabase (local copy in Docker for development)

Smart contract	Solidity `CredentialRegistry` on Ethereum Sepolia, built and tested with Hardhat

2. System architecture

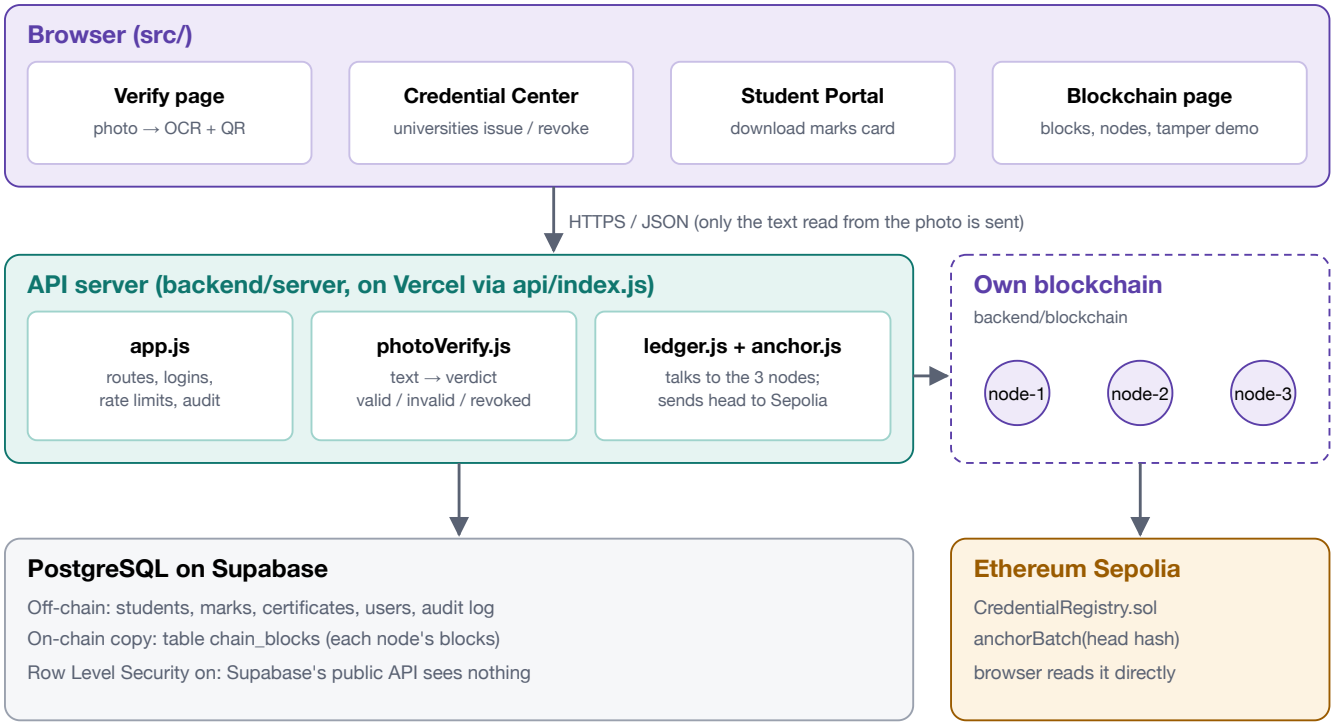


Figure 1. The browser reads the photo and calls the API. The API checks the three blockchain nodes and the database. The newest block's hash is also written to a public smart contract.

What is stored where

Data	Where	Why
Certificate number, SHA-256 of the certificate data, SHA-256 of the marks, issuing university's address, revoked or not	Own blockchain (all 3 nodes)	Cannot be changed without breaking hash links and signatures
Names, dates of birth, marks, programs, users, passwords (hashed), audit log	PostgreSQL tables	Personal data stays off the chain
Hash of the newest block	Solidity contract on Sepolia	A public copy our server cannot rewrite
The certificate photo	Nowhere: it stays in the browser	Privacy; only the text read from it is sent

The fingerprint (why we never hash the photo)

Two photos of the same paper have different pixels, so their hashes never match. We hash the certificate's *data* instead, written in one fixed order:

`cert_no | reg_no | NAME | dob | program_code | score | year | univ_id` → SHA-256

The semester table gets its own hash (`marksHash`), so changing a single SGPA is caught too. Code: `src/lib/certificateView.js` (shared by browser, server and import).

3. Folder map

Folder	What is inside
`backend/blockchain/`	**The blockchain itself**: blocks, transactions, Merkle tree, signatures, nodes, consensus
`backend/server/`	**The API**: web routes, photo verification, link to the chain, database, logins
`backend/server/migrations/`	Database tables as SQL files, applied in number order
`backend/contracts/`	The Solidity smart contract deployed on Sepolia
`backend/deployments/`	The contract's address and interface on Sepolia
`backend/data/karnataka/`	The dataset and (not committed) the test logins
`backend/scripts/`	Commands: import, reset, chain demo, repair, anchor, evaluations, deploy
`backend/test/`	Automated tests (94 run without a database, 6 more with one)
`backend/results/`	Measured accuracy reports
`backend/certs/`	Supabase's security certificate for the encrypted database connection
`src/pages/`	**Website pages**: verify, Credential Center, Student Portal, Blockchain, profile, registration
`src/components/`	Reusable pieces of the pages (photo tab, results, panels, navigation)
`src/lib/`	Website logic: API calls, marks card drawing, photo reading, fingerprint rules
`src/context/`	Shared state: who is signed in, pop-up messages
`public/`	Files served as-is: six sample photos, OCR engine files, icons
`api/`	Vercel's entry point that starts the API server online
`docs/`	Viva guide and this file guide
`archive/`	Old and unused files, kept for reference only (see section 25)

> For the review, open these three: `backend/blockchain/` (how blocks work), `backend/server/photoVerify.js` (photo to verdict), `backend/contracts/CredentialRegistry.sol` (smart contract).

4. Workflow A: loading the dataset onto the chain

Command: `npm run db:reset -- --confirm` then `npm run import:karnataka` (in `backend/`).

1. All application data is deleted; the tables stay.
   scripts/reset-db.js
2. The dataset files are read: universities, colleges, programs, students, enrollments, semester results, degree certificates, NAD awards and verification requests.
   scripts/import-karnataka.js reads data/karnataka/*.json
3. Block 0 (genesis) is created: chain name, the 3 validators, the authority address, difficulty.
   server/ledger.js → blockchain/network.js
4. Each university gets its own signing key and a `SET_ISSUER` transaction authorises it (blocks 1 to 16).
   server/institutions.js, server/wallets.js
5. Each certificate becomes an `ISSUE` transaction signed by its university: certificate number, data hash and marks hash. At most 25 per block (blocks 17 to 88).
   src/lib/certificateView.js computes the hashes
6. Revoked, withheld and cancelled certificates get a `REVOKE` transaction with a hash of the reason (blocks 89 to 105).
7. The academic records go into the database tables; 5 admin and 15 student logins are created and written to `data/karnataka/TEST_LOGINS.md`.
8. `npm run chain:checkpoint` writes the newest block's hash to Sepolia.
   scripts/chain-checkpoint.js → server/anchor.js

Result

Blocks	Contents
0	Genesis: settings, validators, authority
1–16	Authorising universities (`SET_ISSUER`)
17–88	1,604 certificates (`ISSUE` , up to 25 per block)
89–105	86 revocations (`REVOKE`)
106 blocks	**1,707 transactions**, stored 3 times (one copy per node) in `chain_blocks`

5. Workflow B: how one block is created (consensus)

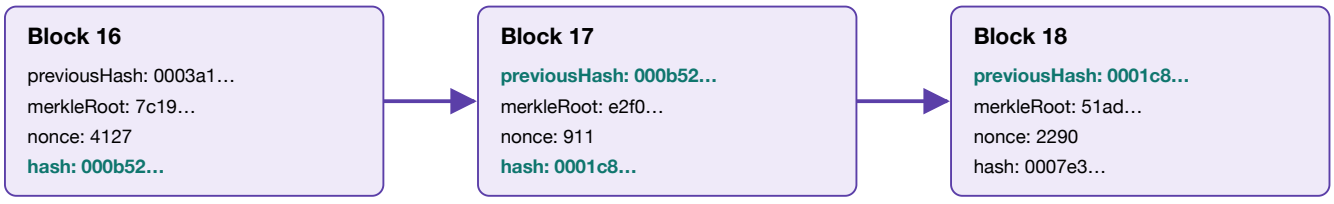


Figure 2. Each block stores the previous block's hash (illustrative values). Changing anything in block 16 changes its hash, so block 17's link breaks.

1. Transactions are signed by their sender (ECDSA secp256k1). Each has an id = SHA-256 of its contents.
   blockchain/transaction.js, blockchain/primitives.js

2. The proposer for block *n* is node *n* mod 3, skipping offline nodes.
   blockchain/network.js
3. It checks every transaction against the rules (authorised issuer, no duplicates, revoke only your own, not paused).
   blockchain/state.js
4. It builds the Merkle root of the transaction ids and mines: tries nonces until the hash starts with 3 zeros.
   blockchain/merkle.js, blockchain/block.js
5. Every node re-checks the block itself (links, Merkle root, proof of work, signatures, rules) and signs its hash as a vote.
   blockchain/node.js
6. With 2 of 3 votes the block is committed. Each node stores its own copy in the database.
   server/ledger.js → table chain_blocks
7. The new head is sent to Sepolia (best effort, never blocks issuing).
   server/anchor.js

6. Workflow C: issuing and revoking a certificate

1. A university registrar signs in to the Credential Center (`/admin`).
   src/pages/AdminConsole.jsx → POST /api/auth/login → server/auth.js
2. They fill in one certificate, or upload a CSV for many.
   AdminConsole.jsx, components/BatchIssuePanel.jsx
3. The server checks the role and that the registrar's university is approved, saves the record and builds the fingerprint.
   server/app.js (POST /api/credentials or /api/credentials/batch), server/store.js
4. An `ISSUE` transaction signed with that university's own key goes through Workflow B.
   server/ledger.js bridge.issue / issueMany
5. The action is written to the audit log.
6. **Revoke:** POST `/api/credentials/:id/revoke` adds a `REVOKE` transaction. Nothing is deleted: issue and revoke both stay visible on the chain. A university can revoke only its own certificates.
7. **Reissue (correction):** revoke the old one and issue a new one in a single step (`/api/credentials/:id/reissue`).

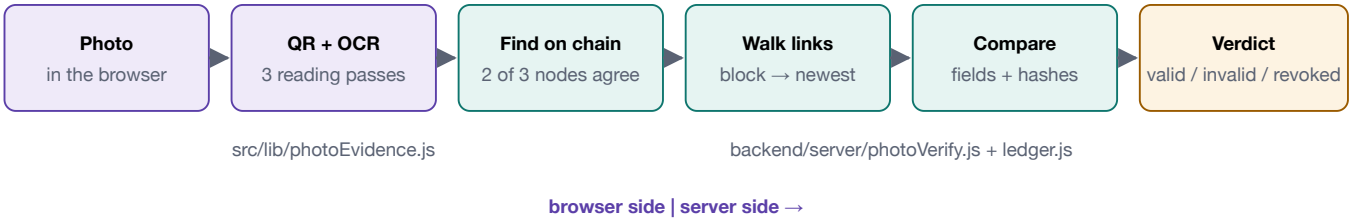
7. Workflow D: a student downloads the marks card

1. The student signs in at `/student` with their personal email. They can only sign up with an email their university has on record.
   components/StudentAuth.jsx → /api/auth/register/student, /api/auth/login
2. The portal asks for their certificates. For each one the server returns the printable view, the hash, transaction id, block number, status and whether the marks hash matches the chain.
   pages/StudentPortal.jsx → GET /api/me/certificates → server/app.js printableCard()
3. A university can download the same card for its own certificates.
   GET /api/credentials/:id/certificate
4. The browser draws the marks card (purple design: student panel, semester table, CGPA tiles, blockchain panel, QR code, verification link, signature) and saves it as an image.
   src/lib/certificateSvg.js
5. The QR code holds a verification link with the certificate number and hash. "Verified" is printed only when the chain confirms the record.
   src/lib/certificateView.js qrPayload / verifyUrl

> The card says "Student project prototype using synthetic data. Not an official university document." It must stay, because the data is synthetic.

8. Workflow E: verifying a certificate photo

This is the project's main output. Page: `/verify` → "Certificate Photo" tab.



1. **Read the photo in the browser.** The image is enlarged to at least 2,000 px wide; the QR code is decoded; Tesseract reads the text three times (whole page, block layout, and a digits-only pass for the marks table), with automatic rotation correction. The photo is never uploaded.
   src/lib/photoEvidence.js, components/PhotoVerifyTab.jsx
2. **Send the text.** POST `/api/verify/photo` with the OCR text and QR text.
3. **Find the certificate number** from the QR, else from the text (allowing common OCR slips such as O/0).
   photoVerify.js extractCertificateNo, prefixCandidates
4. **Ask every node.** Only healthy nodes vote; at least 2 must give the same answer, otherwise the chain is reported unavailable. Not found → **INVALID**.
   ledger.js crossCheck
5. **Walk the chain** from the certificate's block to the newest block, re-checking every hash link and validator signature. Broken → **INVALID**.
   ledger.js walkFrom
6. **Issuer check.** The signing university must be authorised and active. Otherwise → **INVALID**.
7. **Compare printed fields** with the university's record: name (90% similarity allowed for OCR), register number (1 character allowed), date of birth, CGPA, year, program (exact). Every semester row must be read; an unreadable row gives INVALID, never VALID.
   photoVerify.js parseCertificateText, parseSemesterRows, compareFields
8. **Hash checks.** The record must hash to the value on the chain, the marks must hash to the chain's marks hash, and a QR hash (if read) must match.
9. **Status.** Revoked on the chain → **REVOKED** with the reason. All passed → **VALID**.
10. The result shows a proof: nodes that agree, links checked, signatures valid, Merkle proof, transaction and block.
    components/PhotoVerifyTab.jsx

Result codes

Code	Verdict	Meaning
`valid`	VALID	Every detail matches and the hash is on the chain
`revoked`	REVOKED	Genuine, but the university revoked it
`unreadable`	INVALID	No certificate number could be read
`not_on_chain`	INVALID	The certificate number is not on the blockchain (fake serial)
`chain_broken`	INVALID	A hash link or signature after the certificate's block fails

`untrusted_issuer`	INVALID	The signing university is not authorised
`record_missing`	INVALID	On the chain, but the university's record is missing
`fields_unreadable`	INVALID	Some required detail (often a semester row) could not be read: retake the photo
`field_mismatch`	INVALID	A printed detail differs from the record (forged name, CGPA, SGPA, year)
`hash_mismatch`	INVALID	The database record no longer hashes to the chain value (database tampered)
`marks_hash_mismatch`	INVALID	The marks in the database no longer match the chain
`qr_hash_mismatch`	INVALID	The QR code's hash differs from the chain

9. Workflow F: tamper detection and repair

On the Blockchain page, "Run the attack" tries to change a certificate in four escalating steps, on a throwaway copy, and shows what the real validation code reports each time. Code: `server/ledger.js simulateTamper`.

Attacker does	What catches it
1. Edits a transaction	Its id no longer equals SHA-256 of its contents
2. Recomputes the id	The university's signature no longer matches; the Merkle root changes
3. Updates the Merkle root and re-mines the block	The next block's previousHash no longer matches
4. Re-mines every later block	The validators never signed those new hashes

Audit and repair. `GET /api/chain/audit` re-validates every node from block 0. A corrupted node is excluded from voting and can be rebuilt from the healthy majority with `npm run chain:repair`.

10. Workflow G: public anchor on Ethereum Sepolia

1. After a block is committed (or when you run `npm run chain:checkpoint`), the newest block's hash and block count are sent to `CredentialRegistry.anchorBatch` on Sepolia.
   server/anchor.js, scripts/chain-checkpoint.js
2. The Blockchain page reads the anchored hash straight from Sepolia using a public node, without going through our server, and compares it with the chain.
   src/lib/publicAnchor.js, pages/ChainExplorer.jsx
3. If someone rewrote all three copies of our chain, the hashes would no longer match.

> The anchor detects a rewrite made after it; it cannot stop one, and blocks added since the last anchor are not covered. Latest anchor: block 105, transaction 0x9079ebf9…161973c on Sepolia, 2 November 2026.

11. Workflow H: sign-in and accounts

Account	Can do	Where
Platform admin (`platform.admin`)	Approve or suspend universities, pause the registry, read the audit log. Cannot see or issue any university's certificates.	components/PlatformConsole.jsx, InstitutionsPanel.jsx, SecurityPanel.jsx
University registrar	Issue, revoke and view only their own university's certificates	pages/AdminConsole.jsx
Student	See and download only their own certificates; share a public profile	pages/StudentPortal.jsx
Anyone (employer)	Verify by photo, QR, ID or file. No sign-in needed.	pages/VerificationUI.jsx
New institution	Registers; stays pending until the platform admin approves; then gets its own signing key	pages/InstitutionRegister.jsx

Passwords are hashed with scrypt (`server/passwords.js`). A sign-in returns a JWT token checked on every request with the user's role (`server/auth.js`). Repeated failed sign-ins lock the account for a while. Every important action is written to an append-only audit log.

12. Files: the blockchain (`backend/blockchain`)

About 560 lines, written from scratch. It does not depend on Ethereum or any other chain.

File	Lines	What it does
`primitives.js`	36	Cryptographic building blocks: `sha256`, `canonicalJson` (same data always gives the same bytes), ECDSA secp256k1 `signDigest` / `recoverSigner`, addresses, the all-zero hash used by genesis.
`transaction.js`	33	One signed statement. Types: `GENESIS`, `SET_ISSUER`, `ISSUE`, `REVOKE`, `SET_PAUSED`. `transactionId` = SHA-256 of the contents; `checkTransaction` verifies the id and the sender's signature. No personal data.
`merkle.js`	46	Merkle tree over a block's transaction ids: `merkleRoot`, `merkleProof` (about log2(n) sibling hashes), `verifyMerkleProof`.
`block.js`	46	A block = header + transactions + validator signatures. Header: index, previousHash, merkleRoot, timestamp, txCount, difficulty, nonce, proposer. `hashHeader`, `mine` (proof of work), `meetsDifficulty`, `createBlock`, `checkBlockContents`.
`state.js`	141	World state derived only by replaying transactions from block 0: who is authorised, every certificate's hash, marks hash, issuer and status. Enforces the rules and throws `ChainRuleError` (duplicate, unauthorised issuer, revoking twice or another university's certificate, paused).
`node.js`	132	A validator node with its own copy of the chain. `replayChain` re-checks every block (links, Merkle root, proof of work, signatures, rules). `quorumOf` = floor(n/2)+1, `checkVotes`.
`network.js`	125	Consensus among the nodes: round-robin proposer, mining, independent re-check, votes, commit with a majority (`propose`, `commit`). Also `audit` and `repair`. At most 25 transactions per block.
`index.js`	7	Re-exports the files above so other code can import them from one place.

13. Files: the API server (`backend/server`)

File	Lines	What it does
`app.js`	961	All web routes (section 20): logins, issuing, revoking, verifying by ID, file, hash and photo, the chain explorer, admin controls. Adds security headers, rate limits, size limits, JSON errors and the audit log. `printableCard` builds the marks card data.
`photoVerify.js`	279	Photo text → verdict (Workflow E). Reads labelled fields and semester rows, finds the certificate number, compares fields with tolerance for OCR, checks hashes and status, returns a code and reason.
`ledger.js`	525	Connects the blockchain to the API. Stores each node's blocks (memory or PostgreSQL, with a lock so two requests cannot both build the same block), derives validator keys, provides the issue/revoke "bridge", majority reads (`crossCheck`), the chain walk (`walkFrom`), the explorer views and the tamper simulation.
`anchor.js`	40	Writes the newest block's hash to the Solidity contract on Sepolia (`anchorBatch`). Best effort: gives up after 8 seconds and never blocks issuing.
`runtime.js`	63	Builds the complete app from configuration: database store, ledger, anchor, institutions, logins. Shared by the local server and Vercel.
`index.js`	34	Starts the API on your computer (`npm start`), on port 4000.
`config.js`	132	Reads and checks settings from `.env` . In production it refuses to start with unsafe settings (missing secrets, default passwords).
`store.js`	531	The database layer: students, certificates, marks, users, institutions, audit log. A PostgreSQL version and an in-memory version (for tests) with the same functions. Applies migrations.
`auth.js`	102	Sign-in, JWT tokens and roles (platform admin, university registrar, student). Locks an account after repeated failures.
`passwords.js`	33	Password hashing with scrypt (slow on purpose, so stolen hashes are hard to crack).
`institutions.js`	145	Institution lifecycle: register, approve, suspend. Approval gives the university its own signing key and authorises it on the chain.
`wallets.js`	36	Creates each university's signing key and stores it encrypted (AES-256-GCM); it is decrypted only in memory when signing.
`chain.js`	239	From the earlier version: the connection to the Solidity contract with ethers.js. Today its two error types (chain unavailable, rule rejected) are shared by the API, and the contract tests use it.
`merkle.js`	48	From the earlier version: a keccak256 Merkle tree compatible with the Solidity contract, used by the batch-issue route and its tests.

14. Files: database migrations (`backend/server/migrations`)

Applied once each, in number order, by `store.js` (recorded in `schema_migrations`).

File	What it creates
`001_init.sql`	Credentials, users and the audit log

`002_lock_down_public_api.sql`	Turns on Row Level Security and removes access for Supabase's public API roles, so only our server can read the data
`003_institutions_and_students.sql`	Institutions (pending/approved/suspended) and students with personal email
`004_academic_registry.sql`	Full academic registry: colleges, programs, enrollments, semester results, degree certificates, NAD awards, verification requests
`005_own_blockchain.sql`	`chain_blocks` : each node's copy of every block

15. Files: scripts (`backend/scripts`)

File	Lines	What it does	Command
`import-karnataka.js`	452	Loads the dataset into the database and onto the chain (Workflow A); creates test logins.	`import:karnataka`
`reset-db.js`	34	Deletes all application data, including chain blocks; keeps the tables. Needs `--confirm`.	`db:reset`
`db-check.js`	63	Tests the database connection, applies pending migrations, shows row counts and RLS status. Prints no passwords.	`db:check`
`chain-demo.js`	83	Shows the blockchain working in the terminal: blocks, links, votes, a tamper attempt. Runs in memory.	`chain:demo`
`chain-repair.js`	16	Rebuilds a corrupted or lagging node from the healthy majority.	`chain:repair`
`chain-checkpoint.js`	19	Writes the newest block's hash to Sepolia and prints the Etherscan link.	`chain:checkpoint`
`evaluate-verification.js`	80	Runs the 666 labelled verification requests through the real verifier and chain code; writes results/verification-evaluation.md.	`chain:evaluate`
`make-sample-photos.js`	165	Draws marks cards from the dataset, degrades them like phone photos, reads them with the same OCR as the website, measures the verdicts; writes results/photo-ocr.md and the six sample photos in public/samples.	`chain:photos`
`lib/dataset-ledger.js`	73	Builds the dataset's chain in memory, for the two evaluation scripts.	(helper)
`deploy.js`	39	Deploys the Solidity contract and writes deployments/<network>.json.	`deploy:sepolia`
`create-user.js`	50	Creates a user account; asks for the password without showing it.	`create-user`
`vercel-env.js`	56	Copies production settings from backend/.env to Vercel without printing them.	node backend/scripts/vercel-env.js

16. Files: tests and results

File	Lines	What it tests
`test/ownchain.test.js`	376	The own blockchain: Merkle tree, block structure and links, rules, consensus and votes, tampering at each step, photo text parsing and verdicts, chain API routes
`test/security.test.js`	294	Password hashing, sign-in lockout, tokens, security headers, size limits, audit log, emergency pause, contract ownership, startup checks

`test/institutions.test.js`	246	Institution registration and approval, issuing as an institution, student accounts
`test/scenarios.test.js`	210	The 10 Review-1 scenarios end to end (valid, modified, unknown, revoked, untrusted, duplicate, non-admin, chain down, bad upload, correction)
`test/batch.test.js`	170	Batch issuance with Merkle proofs
`test/postgres.test.js`	142	Against a real PostgreSQL (skipped unless a test database is running)
`test/CredentialRegistry.test.js`	126	The Solidity contract on Hardhat's built-in test network
`test/helpers.js`	59	Shared setup: a test app with an in-memory database and two accounts
`results/verification-evaluation.md`	30	Report: 666 labelled requests, all correct
`results/photo-ocr.md`	32	Report: simulated photo accuracy by condition (section 23)

Run with `npm test` in `backend/` : 94 passing, 6 database tests skipped.

17. Files: smart contract, data and settings

Smart contract

File	Lines	What it does
`contracts/CredentialRegistry.sol`	226	Solidity registry on Sepolia. Today the running system uses `anchorBatch(root, count)` to store the head hash of our chain. It also has the earlier per-credential functions (issue, revoke, verify), issuer authorisation, emergency pause (OpenZeppelin Pausable) and two-step ownership.
`deployments/sepolia.json`	859	Contract address on Sepolia and its interface (ABI). The live site needs it.
`hardhat.config.cjs`	20	Solidity compiler version and the Sepolia network settings for Hardhat.

Dataset (`backend/data/karnataka`)

File	Records	Contents
`universities.json`	17	Universities: name, code, type, city, grading system
`colleges.json`	23	Affiliated colleges
`programs.json`	162	Degree programs, branches, durations, credits
`students.json`	2,000	Synthetic students (Aadhaar and mobile masked)
`enrollments.json`	2,000	Which student studies which program where
`semester_results.json`	10,407	Semester-wise credits, SGPA, backlogs, result
`degree_certificates.json`	1,604	Certificates with canonical string, SHA-256 and status
`nad_award_export.json`	1,179	DigiLocker NAD award format
`verification_requests.json`	666	Labelled verification requests with expected results (ground truth)
`data_dictionary.json`	17	Meaning of every column
`karnataka_degree_verification_dataset.xlsx`	—	The same dataset as one Excel workbook
`SOURCE.md`	—	Where the dataset comes from
`TEST_LOGINS.md`	21	Admin and student logins created by the import. **Not committed; do not share.**

Backend settings

File	What it does
`backend/.env`	Secrets: database URL, signing key, JWT secret, encryption key. **Never commit or share.**
`backend/.env.example`	The same settings with no real values, as a template
`backend/package.json`	Backend dependencies and the npm commands (section 22)

`backend/docker-compose.yml`	A local PostgreSQL in Docker on port 5544, for development and database tests
`backend/certs/prod-ca-2021.crt`	Supabase's certificate authority, so the database connection is encrypted and verified
`backend/.gitignore`	Keeps .env, certificates, logins and generated folders out of git

18. Files: the website (`src`, `public`, `api`)

Pages (`src/pages`)

File	Address	What it does
`VerificationUI.jsx`	/verify	The verifier's page. Tabs: certificate photo (default), credential ID, file upload and QR scan. Shows the result, proof and this browser's recent checks.
`ChainExplorer.jsx`	/chain	Blockchain page: totals, the 3 nodes and their health, re-validate, Sepolia anchor check, block list and block details, tamper demo.
`AdminConsole.jsx`	/admin	Credential Center: sign-in, issue one or many, list, revoke, reissue and download certificates. The platform admin sees the platform console instead.
`StudentPortal.jsx`	/student	Student sign-in and sign-up, own certificates, marks card download, share profile.
`PublicProfile.jsx`	/profile/:id	Public "Verified Professional Passport" a student can share.
`InstitutionRegister.jsx`	/register/institution	A new college or issuer registers and waits for approval.

Logic (`src/lib`)

File	What it does
`certificateView.js`	What is printed on a certificate and the exact string that is hashed; QR contents and verification link. Shared with the server.
`certificateSvg.js`	Draws the purple marks card and saves it as an image.
`photoEvidence.js`	Reads a photo in the browser: enlarge, QR decode, three OCR passes with rotation correction.
`publicAnchor.js`	Reads the anchored hash from Sepolia directly and compares it with the chain.
`api.js`	Chooses the data source: the real server when `VITE_API_URL` is set, otherwise the in-browser mock.
`httpApi.js`	Calls to the real API server.
`mockApi.js`	An in-browser imitation of the server, for a demo without a backend.
`qr.js`	Decides what a scanned QR code means (verification link, credential ID or profile link).
`verification.js`	Turns a result into one of four states and its checklist.
`generateCertificatePdf.js`	Simple PDF certificate used by the mock demo and the file-upload check.
`studentProfile.js, credentialGraph.js`	Build the student profile and credential graph views from the student's own records.
`institution.js`	The reference institution's name in one place.

Components (`src/components`)

File	What it does
`PhotoVerifyTab.jsx`	Upload or photograph a certificate; six sample buttons; progress; verdict with proof (nodes agree, links checked, signatures, marks hash).
`QrScanTab.jsx`	Scan a QR code with the camera.
`VerificationResult.jsx, VerificationStateBanner.jsx, VerificationChecks.jsx`	Show a verification result: headline, explanation and checklist (never colour alone).
`VerificationProofModal.jsx`	Recomputes a certificate's hash and compares it with the record.
`VerificationHistory.jsx`	Recent checks made in this browser only.
`OnChainHistory.jsx`	Issue and revoke history of one certificate.
`PlatformConsole.jsx, InstitutionsPanel.jsx, SecurityPanel.jsx`	Platform admin: approve or suspend institutions, pause the registry, audit log.
`BatchIssuePanel.jsx`	Issue many certificates from a CSV file.
`StudentAuth.jsx`	Student sign-in and sign-up with a personal email.
`ShareProfileModal.jsx, AcademicJourney.jsx, CredentialGraph.jsx, CredentialDNA.jsx`	Student profile sharing, timeline, graph and a visual hash fingerprint.

`TopNav.jsx, PrototypeBanner.jsx`	Top navigation (Verify, Student, Credential Center, Blockchain) and the "prototype with synthetic data" banner.
`HashChip.jsx, StatusBadge.jsx, Spinner.jsx`	Small pieces: a shortened hash with copy, a status badge, a loading spinner.

Other website files

File	What it does
`src/main.jsx, src/App.jsx`	Start React and define which page shows at which address.
`src/context/AuthContext.jsx, ToastContext.jsx`	Who is signed in; pop-up messages.
`src/index.css, index.html`	Styles (Tailwind) and the page shell.
`public/samples/*.jpg`	The six demo photos: genuine, revoked, forged CGPA, forged SGPA, forged name, fake serial.
`public/tesseract/`	The OCR engine and English language data, served from our own site.
`api/index.js`	Starts the API as a Vercel serverless function; every `/api/*` request comes here.

19. Root files

File	What it does
`README.md`	Project overview, folder map, how to run
`SECURITY.md`	Threat model and every security control
`package.json`	Website dependencies and commands: `dev`, `build`, `lint`, `preview`
`vite.config.js`	Settings for Vite, which runs and builds the website
`vercel.json`	Sends `/api/*` to the API function and everything else to the website; includes migrations and the contract address in the upload
`.vercelignore`	Keeps secrets, data, tests, docs and the archive out of the Vercel upload
`.gitignore`	Keeps generated folders and editor files out of git
`.oxlintrc.json`	Settings for the code checker (`npm run lint`)

20. API endpoints

Endpoint	Who	Purpose
`GET /api/health`	Anyone	Server, chain and database status
`POST /api/auth/login`	Anyone	Sign in, returns a token
`POST /api/auth/register/student`	Anyone	Student sign-up (email must be on record)
`POST /api/institutions/register`	Anyone	New institution registration
`GET /api/institutions`	Anyone	Approved institutions
`POST /api/verify/photo`	Anyone	Text read from a photo → verdict
`GET /api/verify/:credentialId`	Anyone	Verify by ID
`POST /api/verify/hash, /api/verify/file`	Anyone	Verify by hash or uploaded file
`GET /api/chain/summary, /blocks, /blocks/:index`	Anyone	Blockchain explorer
`GET /api/chain/audit`	Anyone	Re-validate every node from block 0
`POST /api/chain/simulate-tamper`	Anyone	Tamper demo on a throwaway copy
`GET /api/me`	Signed in	Own account
`GET /api/me/certificates, /api/me/credentials`	Student	Own certificates with marks card data
`GET /api/credentials/:id/certificate`	University	Marks card data for its own certificate
`GET /api/credentials/:id/history`	Anyone	Issue and revoke history
`POST /api/credentials, /api/credentials/batch`	University	Issue one or many
`GET /api/credentials`	University	List own certificates
`GET /api/credentials/:id`	Anyone	One credential's public record

`POST /api/credentials/:id/revoke, /reissue`	University	Revoke or correct
`GET /api/students/:studentId/credentials`	Anyone	Credentials shown on a student's public profile
`GET, POST /api/admin/institutions…`	Platform admin	Approve, reject, suspend institutions
`GET /api/admin/audit, /api/admin/registry`	Platform admin	Audit log and registry status
`POST /api/admin/pause`	Platform admin	Emergency pause or resume

21. Database tables

Table	Holds
`chain_blocks`	Every block, once per node (node id, index, hash, previous hash, the full block)
`credentials`	Issued certificates linked to the chain: id, certificate number, hash, transaction, block, status
`institutions`	Universities and issuers: status, encrypted signing key, address, details
`users`	Accounts: username, scrypt password hash, role, institution, lockout counters
`students`	Student identity (masked Aadhaar and mobile), personal email
`colleges, programs, enrollments`	Where and what each student studies
`semester_results`	Semester-wise credits, SGPA, backlogs, result
`degree_certificates`	Full certificate records with canonical string and hash
`nad_awards`	DigiLocker NAD export format
`verification_requests`	Labelled test requests with expected results
`audit_log`	Append-only record of logins, issues, revocations, approvals, pauses
`schema_migrations`	Which migration files have been applied

Row Level Security is on for every table, and Supabase's public API roles have no access. Only the API server, connecting as the table owner, can read or write.

22. Commands you can run

Command (in `backend/`)	What it does
`npm test`	Run the automated tests
`npm run test:db`	Tests including the PostgreSQL ones (needs `db:up`)
`npm start`	Start the API on http://localhost:4000
`npm run db:up / db:down`	Start or stop the local PostgreSQL in Docker
`npm run db:check`	Check the database connection and apply migrations
`npm run db:reset -- --confirm`	Delete all data (keeps tables)
`npm run import:karnataka`	Load the dataset onto the chain
`npm run chain:demo`	Show the blockchain working in the terminal
`npm run chain:repair`	Repair a corrupted node
`npm run chain:checkpoint`	Anchor the newest block on Sepolia
`npm run chain:evaluate`	Run the 666 labelled requests
`npm run chain:photos`	Measure photo accuracy (takes about 10 minutes)

`npm run compile`	Compile the Solidity contract
`npm run deploy:sepolia`	Deploy the contract (already done)
`npm run create-user`	Create an account
Command (in the project root)	
`npm run dev`	Run the website locally (add `VITE_API_URL=http://localhost:4000` to use the local API)
`npm run build`	Build the website into `dist/`
`vercel --prod`	Deploy the website and API to Vercel

23. Measured results

Test	Result
Automated tests	94 passing; 6 database tests skipped without a database
Labelled verification requests (clean text)	666 / 666 correct
Simulated marks-card photos, overall	120 / 126 correct
Forged or fake photos rejected	90 / 90
Genuine or revoked photos read correctly	30 / 36 (clean 11/12, phone-like 11/12, poor 8/12)
Six demo samples on the live site	6 / 6 correct
Live chain audit	All 3 nodes valid from block 0 to 105
Live chain head vs. Sepolia anchor	Match

> The 666 requests test the verifier and chain, not OCR. The 126 photos were made and degraded by software, not taken by a camera, and the reading rules were tuned on such images; real-camera accuracy has not been measured and will likely be lower. All 6 photo errors reject a real certificate because a semester row could not be read; a clearer photo fixes that. 90 rejected forgeries in a sample does not prove no forgery can ever pass.

24. Known limits

1. **Not fully decentralised.** The 3 nodes run in one server process with keys derived from one secret and copies in one database. Real decentralisation needs nodes run by different universities on separate machines. Describe it as a hybrid: own chain plus a public anchor on Sepolia, not a full DApp.
2. **Three validators** tolerate one node crashing, not a malicious majority. Classical BFT needs at least 4 nodes to tolerate one malicious node.
3. **No peer-to-peer network.** Blocks are created on request; there is no gossip or shared pool of pending transactions.
4. **OCR tolerance.** Names allow small differences (90% similarity) and register numbers one character, so a one-letter change on a long name may pass. Numbers must match exactly.
5. **Proves the document, not the person.** An exact copy of someone's genuine certificate returns VALID.
6. **Only this system's card layout can be read.** A certificate printed by a real university will not parse.
7. **Synthetic data.** The dataset and the certificates are for a student project, not official documents.

25. Generated folders and the archive

Folder	What it is
`node_modules/`	Downloaded libraries (root and backend). Created by `npm install`.
`dist/`	The built website, created by `npm run build`. Vercel builds its own.
`backend/artifacts/, cache/`	Compiled Solidity contract, created by Hardhat; needed by the tests.
`.vercel/`	Links this folder to the Vercel project.

`archive/review-2-documents/`	Review-2 slides, reports and dataset analysis (older design)
`archive/old-fluree-dataset/`	The first dataset and its import script, replaced by the Karnataka dataset
`archive/old-scripts/`	Old gas benchmark, demo seeding, Sepolia cost check, card preview, old local contract address
`archive/tool-files/`	Leftovers from development tools

Generated folders are hidden in VS Code (`.vscode/settings.json`); they are not deleted, because the tools need them.